



RunPipeline

- collect_brand_data(brand, limit_per_query) : DataFrame
- clean_text(text) : string
- preprocess_data(df, vectorizer) : DataFrame
- predict_sentiment(df) : DataFrame
- summarize_results(df) : DataFrame
- main() : void

imports

imports



Scraper

- string REDDIT_USER_AGENT
- string_token
- float_token_expiry
- _get_oauth_token(client_id, client_secret) : string
- _auth_headers(token) : dict
- _reddit_url(url, token) : string
- build_queries(brand) : list
- is_egypt_relevant(text, brand) : bool
- get_posts(query, limit, token) : list
- get_comments(permalink, query, token) : list
- collect_brand_data(brand, limit, client_id, client_secret) : DataFrame



Recommendation

- dict SENTIMENT_WEIGHTS
- _brand_from_path(path) : string
- build_company_profile(df, brand, source) : dict
- rank_companies(profiles) : DataFrame
- load_processed_company_profiles(data_dir) : DataFrame
- build_ml_flow_summary(raw_rows, predicted_rows, ranked_companies) : DataFrame



PredictReddit

- clean_text(text) : string
- preprocess_data(df, vectorizer) : DataFrame
- main() : void

matches preprocessing parameters



ModelTraining

- Path ROOT_DIR
- Path LABELED_PATH
- Path MODEL_SAVE_PATH
- Path VEC_SAVE_PATH
- clean_text(text) : string
- tokenize(text) : list
- lemmatize_word(word) : string
- preprocess_text(text) : string
- main() : void